

Docker & Kubernetes

The Complete Developer Guide

Docker
Container Engine

Kubernetes
Container Orchestration

1. What is Docker?

Docker is an open-source platform launched in 2013 that allows developers to package applications and all their dependencies into a standardized unit called a **container**. A container is a lightweight, portable, self-sufficient environment that runs the same way regardless of the host machine — whether it's your laptop, a test server, or a cloud data center.

Before Docker, the classic problem was *"It works on my machine"* — code would behave differently across environments due to OS differences, library versions, or configuration mismatches. Docker solves this by bundling everything the app needs into one image.

2. What is Kubernetes?

Kubernetes (often abbreviated **K8s**) is an open-source container orchestration system originally developed by Google, now maintained by the Cloud Native Computing Foundation (CNCF). While Docker creates and runs individual containers, Kubernetes manages hundreds or thousands of containers across multiple machines — automatically scaling, healing, and load-balancing them.

Think of it this way: **Docker is the shipping container; Kubernetes is the entire shipping port** that decides where containers go, replaces broken ones, and routes traffic intelligently.

Why & Where We Use Docker and Kubernetes

3. Why Use Docker?

- **Consistency:** Run the exact same container in dev, staging, and production — zero environment drift.
- **Isolation:** Each container is sandboxed. A crash in one container doesn't affect others.
- **Speed:** Containers start in milliseconds (vs. minutes for VMs), boosting developer productivity.
- **Portability:** A Docker image runs on any OS with Docker installed — Windows, Mac, Linux, cloud.
- **Microservices:** Each service (auth, payments, notifications) lives in its own container, independently deployable.
- **CI/CD Pipelines:** Docker integrates cleanly with Jenkins, GitHub Actions, GitLab CI for automated builds & deploys.
- **Resource Efficiency:** Containers share the host OS kernel — much lighter than full virtual machines.

4. Why Use Kubernetes?

- **Auto-scaling:** K8s automatically scales your containers up/down based on CPU, memory, or custom metrics.
- **Self-healing:** If a container crashes, K8s restarts it automatically. Dead nodes? Containers are rescheduled.
- **Load Balancing:** Distributes incoming traffic evenly across all healthy container instances.
- **Rolling Updates:** Deploy new versions with zero downtime — K8s gradually replaces old containers with new ones.
- **Secret Management:** Safely store and inject passwords, API keys, and certificates into containers.
- **Multi-cloud:** Run the same K8s workload on AWS, GCP, Azure, or on-premise with no code changes.
- **Service Discovery:** Containers automatically find each other by name — no hardcoded IPs needed.

5. Where Are They Used?

Industry	Docker Use Case	Kubernetes Use Case
E-commerce	Package each microservice (cart, search, payments)	Scale checkout service 10x on Black Friday
FinTech	Consistent builds for regulatory compliance	Rolling deploys with zero transaction downtime
Healthcare	Isolate sensitive patient data per container	Auto-heal critical monitoring services 24/7

Gaming	Fast local dev environments for game servers	Spin up game server instances per region/demand
SaaS / Startups	Ship features quickly with Docker Compose	Manage multi-tenant workloads at scale
Data / ML	Reproducible model training environments	Orchestrate GPU-heavy training jobs (Kubeflow)

Core Concepts You Must Know

6. Essential Docker Concepts

Concept	What It Means
Dockerfile	A text file with step-by-step instructions to build a Docker image. Every image starts from a Dockerfile.
Image	A read-only snapshot of your app + its environment. Built from a Dockerfile. Think of it as a template.
Container	A running instance of an image. You can run many containers from the same image simultaneously.
Docker Hub	The public registry (like GitHub for images) where you pull official images (nginx, postgres, node, etc.).
Volume	Persistent storage mounted into a container. Data survives even if the container is deleted.
Network	Docker's virtual network that lets containers communicate with each other securely.
Docker Compose	A YAML file to define and run multi-container apps (e.g., web app + database + cache) with one command.
Registry	A storage system for Docker images. Can be Docker Hub, AWS ECR, Google GCR, or self-hosted.

7. Essential Kubernetes Concepts

Concept	What It Means
Pod	The smallest deployable unit in K8s. A pod wraps one or more containers that share the same network/storage.
Node	A physical or virtual machine in the cluster. Each node runs pods managed by the control plane.
Cluster	A group of nodes managed by Kubernetes together as one logical computing resource.
Deployment	Declares the desired state — e.g., 'I want 5 replicas of my web app running.' K8s maintains it.
Service	A stable network endpoint (with a fixed IP/DNS) that routes traffic to the right pods.
Namespace	A virtual partition of the cluster. Use it to separate dev/staging/prod or different teams.
ConfigMap	Store non-sensitive configuration (env variables, config files) separately from your container image.
Secret	Like ConfigMap but for sensitive data (passwords, tokens). Stored encrypted.
Ingress	Manages external HTTP/HTTPS access to services — like a smart reverse proxy inside your cluster.
kubectl	The command-line tool to talk to your Kubernetes cluster. Apply configs, check logs, scale apps.

Commands, Architecture & YAML

8. Must-Know Docker Commands

Command	What It Does
<code>docker build -t myapp:1.0 .</code>	Build an image from Dockerfile in current directory, tagged 'myapp:1.0'
<code>docker run -d -p 8080:80 myapp:1.0</code>	Run container in background, map host port 8080 to container port 80
<code>docker ps</code>	List all running containers
<code>docker logs <container_id></code>	View logs from a running container
<code>docker exec -it <id> bash</code>	Open an interactive shell inside a running container
<code>docker stop <container_id></code>	Gracefully stop a running container
<code>docker pull nginx:latest</code>	Download the latest nginx image from Docker Hub
<code>docker push myrepo/myapp:1.0</code>	Push your image to a Docker registry
<code>docker compose up -d</code>	Start all services defined in docker-compose.yml in background
<code>docker compose down</code>	Stop and remove all containers defined in docker-compose.yml
<code>docker images</code>	List all locally available Docker images
<code>docker rmi <image_id></code>	Remove a local Docker image

9. Must-Know kubectl Commands

Command	What It Does
<code>kubectl get pods</code>	List all pods in the current namespace
<code>kubectl get nodes</code>	List all nodes in the cluster
<code>kubectl apply -f deployment.yaml</code>	Create/update resources defined in a YAML file
<code>kubectl describe pod <name></code>	Show detailed info about a pod (events, errors)
<code>kubectl logs <pod-name></code>	Print logs from a pod's container
<code>kubectl scale deploy myapp --replicas=5</code>	Scale a deployment to 5 replicas
<code>kubectl delete pod <name></code>	Delete a pod (K8s will restart it via Deployment)
<code>kubectl get services</code>	List all services in the namespace

<code>kubectl rollout status deploy/myapp</code>	Watch the rollout status of a deployment
<code>kubectl rollout undo deploy/myapp</code>	Roll back to the previous deployment version

Learning Roadmap & Sample YAML

10. Sample Dockerfile

Every Docker journey starts with writing a Dockerfile. Here's a real-world Node.js example:

```
FROM node:20-alpine # Start from official lightweight Node image
WORKDIR /app # Set working directory inside container
COPY package*.json ./ # Copy dependency files first (for cache efficiency)
RUN npm install # Install dependencies
COPY . . # Copy the rest of the app source code
EXPOSE 3000 # Document the port the app listens on
CMD ["node", "server.js"] # Command to run when container starts
```

11. Sample Kubernetes Deployment YAML

A Deployment YAML tells Kubernetes exactly what to run and how many copies to maintain:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3 # Run 3 identical pods
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels: {app: myapp}
    spec:
      containers:
        - name: myapp
          image: myrepo/myapp:1.0 # Docker image to use
          ports:
            - containerPort: 3000
```

```
resources:
limits: {cpu: '500m', memory: '256Mi'}
```

12. Learning Roadmap

Stage	Topic	Tools / Resources
1 — Foundations	Linux basics, CLI, networking (ports, DNS, TCP/IP)	TCPIP Journey, TryHackMe
2 — Docker Basics	Install Docker, write Dockerfiles, build & run Docker images	Docker Docs, Play With Docker
3 — Docker Advanced	Docker Compose, volumes, networks, multi-stage builds	docker-compose.yml labs
4 — Push to Registry	Tag & push images to Docker Hub or AWS ECR	Docker Hub account, AWS free tier
5 — K8s Intro	Pods, Deployments, Services, kubectl basics	Minikube, K3s, Killercodea
6 — K8s Intermediate	ConfigMaps, Secrets, Ingress, Namespaces, RBAC	Kubernetes official docs
7 — K8s Advanced	Helm charts, Operators, HPA, resource limits	Helm Docs, Artifact Hub
8 — Production	CI/CD pipelines, monitoring (Prometheus+Grafana), logging	GitLab, ArgoCD
9 — Cloud K8s	EKS (AWS), GKE (Google), AKS (Azure) managed providers	Cloud providers free tiers
10 — Certifications	CKA (Certified Kubernetes Administrator)	killer.sh practice exams

Pro Tip: Start with Docker, get comfortable, then move to Kubernetes. Don't try to learn both simultaneously. Master docker build, run, compose first — K8s will feel much more natural once you think in containers.